Instagram clone

A complete build reference — frontend, backend, database, real-time, and UI, mapped to how the app actually gets built

Next.js · NestJS · PostgreSQL · Prisma · Redis · AWS S3
Tailwind CSS · shadcn/ui · Socket.io · WebRTC

Prepared for Hardik Rastogi

How to read this document

This document walks through the Instagram clone in the order it actually gets built and used: login first, then the home feed, stories, explore, notifications, profile, chat, and video calling — followed by the cross-cutting infrastructure and design system that support all of it.

Every term, library, or component follows the same four-part structure:

Field	What it means
1. Definition	A one- or two-line explanation of what it actually is.
2. Used in	The specific page, module, or file where it shows up.
3. Why	The reason this piece exists — what problem it solves.
4. Importance	Essential / Important / Good to know / Optional — a plain-spoken read on whether this can be skipped for a v1.

This PDF's outline/bookmarks panel (in any PDF reader) is set up with every section and term as a jump-to entry, so it can be used as a searchable reference later, not just read top to bottom.

1. Tech stack at a glance

The full picture in one table, before the section-by-section detail begins.

Layer	Technology	Role
Frontend framework	Next.js (App Router, TypeScript)	Pages, routing, rendering
Backend framework	NestJS (TypeScript)	REST API + WebSocket gateways
Database	PostgreSQL	Primary data store
ORM	Prisma	Schema, migrations, type-safe queries
Cache / live state	Redis	Feed cache, chat presence
Object storage	AWS S3	Photos and videos
Real-time transport	Socket.io (via NestJS gateways)	Chat, notifications, call signaling
Video calling	WebRTC + STUN/TURN	Peer-to-peer audio/video
Auth	JWT + Passport.js + Google OAuth	Login, sessions, Google sign-in
Styling	Tailwind CSS + shadcn/ui	Design system and components
Data fetching (frontend)	TanStack Query	Server-state caching
Client state (frontend)	Zustand	UI-only state, e.g. active call
Forms & validation	react-hook-form + zod / class-validator	Input validation, both sides
Dev environment	Docker + docker-compose	Local Postgres/Redis parity
Deployment	Vercel + Render/Railway + managed DB/Redis	Hosting

2. Login & signup

The front door of the app. Two pages on the Next.js side (Login, Signup) talk to one module on the NestJS side (AuthModule). Nothing else in the app is reachable until this works.

Login page & signup page — UI screens

The two entry screens of the app — signup collects a username/email/password or a Google button, login verifies an existing account.

USED IN — Next.js App Router — app/(auth)/login/page.tsx and app/(auth)/signup/page.tsx.

WHY — They're the only pages a logged-out user can reach; every other route redirects here first.

IMPORTANCE — Essential — Can't skip — nothing else works without them.

NestJS AuthModule — Backend module

Groups the AuthController (routes like /auth/login, /auth/signup, /auth/google) and AuthService (checking passwords, creating users, issuing tokens).

USED IN — server/src/auth/ — auth.controller.ts, auth.service.ts, auth.module.ts, plus DTOs for each request body.

WHY — Keeps every authentication concern in one place instead of scattered across the app.

IMPORTANCE — Essential — This is the backbone of the whole login flow.

Passport.js strategies (local, JWT, Google) — Library

NestJS's wrapper around Passport, the standard Node.js auth library. Each 'strategy' is a small plugin for one login method.

USED IN — server/src/auth/strategies/ — local.strategy.ts, jwt.strategy.ts, google.strategy.ts.

WHY — Instead of writing three separate auth flows by hand, each strategy plugs into the same Guard system NestJS already provides.

IMPORTANCE — Important — You could hand-roll this, but it's the standard NestJS pattern and worth learning once.

---

bcrypt — Library

A one-way hashing algorithm built for passwords — deliberately slow, which makes brute-forcing expensive.

USED IN — AuthService.signup() before saving a new user; AuthService.validateUser() when checking a login attempt.

WHY — You must never store a plain-text password. bcrypt compares attempts against a hash without ever decrypting anything.

IMPORTANCE — Essential — Non-negotiable for any app that stores passwords.

JWT (JSON Web Token) + @nestjs/jwt — Concept + library

A signed, self-contained token that proves who a user is without the server having to remember them.

USED IN — AuthService issues one after login/signup/Google auth; jwt.strategy.ts verifies it on every protected request.

WHY — Stateless auth — the server checks a signature instead of keeping a session table.

IMPORTANCE — Essential — Core to how every protected route knows who's asking.

---

httpOnly cookie + cookie-parser — Config + library

Instead of handing the JWT to JavaScript (readable, stealable), the backend sets it as a cookie the browser can't script-access.

USED IN — AuthController sets it on response; cookie-parser middleware in main.ts reads it back off requests.

WHY — Meaningfully safer against XSS than storing the token in localStorage.

IMPORTANCE — Important — localStorage would still work — this is just the safer default.

Next.js Middleware — Frontend concept

A function that runs before a page loads, redirecting based on request data — here, whether the auth cookie is present.

USED IN — middleware.ts at the project root, with a matcher for protected routes like /home, /profile, /chat.

WHY — Stops a logged-out user from ever seeing a protected page's HTML.

IMPORTANCE — Important — Good practice, though gating client-side works too in a pinch.

Google Cloud OAuth credentials — Config, no code

A project in Google Cloud Console with an OAuth consent screen and a client ID/secret — what lets your app ask Google 'is this really this person?'

USED IN — Used by google.strategy.ts on the backend, referenced by the 'Continue with Google' button on the frontend.

WHY — Google won't authenticate your users without this one-time registration step.

IMPORTANCE — Essential — Required since Google Sign-In is in scope — setup, not code.

---

class-validator + class-transformer — Library

Decorator-based validation for NestJS DTOs — e.g. @IsEmail(), @MinLength(8) on a SignupDto class.

USED IN — server/src/auth/dto/login.dto.ts and signup.dto.ts.

WHY — Rejects malformed requests before they reach your business logic.

IMPORTANCE — Important — NestJS's standard validation approach — use it consistently, not just in auth.

---

react-hook-form + zod — Library

react-hook-form manages form state with minimal re-renders; zod defines a validation schema that catches errors before a request is sent.

USED IN — Login and signup forms on the Next.js frontend.

WHY — Instant client-side feedback without a round trip to the server.

IMPORTANCE — Important — Pairs naturally with shadcn/ui's Form component, built around this exact combination.

---

shadcn/ui auth components — Component library

Pre-built, accessible pieces — Form, Input, Button, Card, Label, and a toast for errors — copied into your codebase and styled with Tailwind.

USED IN — components/ui/, used directly inside login and signup pages.

WHY — Saves hand-building form inputs, buttons, and error states from scratch.

IMPORTANCE — Good to know — A UI convenience — swappable for hand-rolled components.

---

3. Home feed

Once logged in, the home feed is what a user lands on — a scrolling list of posts from people they follow. This is the core loop of the app.

Home feed page — UI screen

The main scrolling list of posts from followed accounts, newest first.

USED IN — app/(main)/home/page.tsx.

WHY — The default landing screen after login.

IMPORTANCE — Essential

PostsModule (NestJS) — Backend module

Handles everything post-related: creating a post, fetching the feed, deleting a post.

USED IN — server/src/posts/ — posts.controller.ts, posts.service.ts.

WHY — Central place for the feed query and post-creation logic.

IMPORTANCE — Essential

Prisma Post & PostMedia models — Database schema

Post holds the caption/author/timestamps; PostMedia is a separate table (S3 keys) so one post can hold multiple photos or one video.

USED IN — server/prisma/schema.prisma.

WHY — Splitting media out means a post can be a single photo, a carousel, or a video without changing the schema.

IMPORTANCE — Essential

Feed query logic — Concept

A query joining Post to Follow, filtered to accounts the current user follows, ordered by recency, paginated.

USED IN — PostsService.getFeed().

WHY — This one query is the entire 'algorithm' behind the home feed for a v1.

IMPORTANCE — Essential

TanStack Query (React Query) — Library

A data-fetching library that caches server responses, refetches when stale, and gives loading/error states for free.

USED IN — Wraps every API call from the frontend — feed, likes, comments, profile, everywhere.

WHY — Without it you'd hand-write loading spinners, caching, and refetch logic for every screen.

IMPORTANCE — Essential — One of the highest-leverage libraries in this whole stack.

Infinite scroll — Pattern

Loads the feed in pages of ~10-20 posts, fetching the next page as the user nears the bottom.

USED IN — Home feed page, via TanStack Query's useInfiniteQuery plus an IntersectionObserver on the last post.

WHY — Loading the entire feed at once would be slow and wasteful.

IMPORTANCE — Important

Post card component — Frontend component

The reusable card showing one post — avatar, username, media, caption, like/comment buttons.

USED IN — components/post-card.tsx, rendered in the feed, explore grid, and profile grid.

WHY — Written once, reused across three different pages.

IMPORTANCE — Essential

Like button + optimistic updates — Pattern

Clicking like updates the UI instantly, before the server confirms it, then quietly syncs.

USED IN — Post card component, via a TanStack Query mutation with onMutate.

WHY — Makes the app feel instant instead of waiting on a network round trip.

IMPORTANCE — Important — Worth learning — this exact pattern shows up in almost every modern app.

Comment section component — Frontend component

A list of comments under a post, with an input to add a new one.

USED IN — components/comment-section.tsx, opened from the post card.

WHY — Comments are core to the 'like them, comment etc.' requirement.

IMPORTANCE — Essential

---

next/image — Built into Next.js

Next.js's built-in image component — auto-resizes, lazy-loads, serves modern formats.

USED IN — Every post image and avatar across the app.

WHY — A feed of unoptimized full-size images would load painfully slowly.

IMPORTANCE — Important — Requires whitelisting your S3 domain in next.config.js.

AWS S3 presigned upload — Pattern

Instead of a photo/video traveling through NestJS, the backend hands the browser a short-lived signed URL and the browser uploads straight to S3.

USED IN — UploadModule generates the URL; the 'new post' form uses it before saving the post record.

WHY — Keeps large video files off your server entirely — better performance and cost.

IMPORTANCE — Important — A little more setup than a plain file upload, but the standard real-world approach.

Redis feed caching — Pattern

Caching a user's computed feed briefly so refreshing the page doesn't re-run the full query every time.

USED IN — PostsService.getFeed(), checking Redis before hitting Postgres.

WHY — Cuts repeated database load for a query that runs constantly.

IMPORTANCE — Good to know — Skippable for a small portfolio deployment — add once the feed query is genuinely slow.

---

shadcn/ui feed components — Component library

Avatar, Dialog (comment modal), DropdownMenu (post options), Skeleton (loading placeholders).

USED IN — components/ui/, used throughout the feed and post card.

WHY — Consistent, accessible building blocks instead of custom CSS for each one.

IMPORTANCE — Good to know

---

4. Stories

A row of circular avatars above the feed that open into a full-screen, auto-advancing viewer. Content disappears after 24 hours.

Stories bar — UI component

A horizontally scrollable row of avatars at the top of the home feed, one per person with an active story.

USED IN — components/stories-bar.tsx, rendered above the feed.

WHY — This is how a user discovers there's a story to watch.

IMPORTANCE — Essential

Story ring / seen-unseen indicator — UI detail

A gradient ring around an avatar means an unwatched story; a plain gray ring means already seen.

USED IN — Inside the stories bar avatar, a conditional gradient border styled with Tailwind.

WHY — Without this cue, users can't tell what's new at a glance.

IMPORTANCE — Good to know — Visual polish — the app works without it, but it's easy and high-impact.

Story viewer — UI component

A full-screen overlay that plays through a person's stories one at a time, with a segmented progress bar and tap-to-advance.

USED IN — components/story-viewer.tsx, opened when a stories-bar avatar is tapped.

WHY — This is where the actual story content is watched.

IMPORTANCE — Essential

StoriesModule + Story/StoryView models — Backend module + schema

Story holds media and an expiresAt timestamp; StoryView records who's watched which story.

USED IN — server/src/stories/ and the matching models in schema.prisma.

WHY — Almost identical shape to posts, but with an expiry field and a lighter-weight view record.

IMPORTANCE — Essential

24-hour expiry logic — Pattern

Stories older than 24 hours are simply excluded from the query — no need to delete them the instant they expire.

USED IN — StoriesService.getActiveStories(), filtering WHERE expiresAt > now().

WHY — Simplest possible way to implement 'disappearing' content.

IMPORTANCE — Important — An optional cleanup cron (@nestjs/schedule) can delete expired rows and S3 files later — not required for the feature to work.

Story upload flow — Pattern

Reuses the same S3 presigned-upload pattern as regular posts — one photo or short video per story.

USED IN — Story creation form, calling the same UploadModule as posts.

WHY — No need to build a second upload system for stories.

IMPORTANCE — Good to know

---

5. Explore

A search bar up top, and a grid of posts from people the user doesn't follow below it — the discovery surface.

Explore page — UI screen

A grid of post thumbnails from across the platform, with a search bar pinned to the top.

USED IN — app/(main)/explore/page.tsx.

WHY — One of your four required sections — the 'discover new people' surface.

IMPORTANCE — Essential

Search endpoint — Backend route

A route that takes a text query and returns matching users.

USED IN — UsersController's /users/search, using Postgres's ILIKE for a case-insensitive partial match.

WHY — Powers the search bar at the top of Explore.

IMPORTANCE — Essential — Postgres full-text search or Elasticsearch/Algolia are later upgrades — ILIKE is genuinely fine at this scale.

Debounced search input — Frontend pattern

Waits for the user to pause typing (~300ms) before firing the search request, instead of one request per keystroke.

USED IN — Search bar component on the Explore page.

WHY — Stops the search endpoint from being hit on every keystroke.

IMPORTANCE — Important

Explore grid layout — UI pattern

A responsive CSS grid of square thumbnails — 3 columns on mobile, more on desktop.

USED IN — Explore page, using Tailwind's grid utilities directly.

WHY — Matches the dense, scannable browsing experience of the real app.

IMPORTANCE — Good to know

Random / mixed post query — Backend logic

Pulls recent posts from accounts the user does NOT follow, shuffled — no personalized ranking.

USED IN — PostsService.getExploreFeed().

WHY — This is exactly what you described — 'random mixed posts of random people around the world.'

IMPORTANCE — Essential — A real recommendation algorithm is a project of its own — random-but-recent is a completely reasonable v1.

---

6. Notifications

A bell icon with an unread badge, opening a list of 'so-and-so liked your post' updates — delivered live.

Notification bell + badge — UI component

An icon in the nav bar showing an unread count, opening a panel of recent notifications.

USED IN — Part of the main app layout/navbar, visible on every page.

WHY — The primary way users find out someone interacted with them.

IMPORTANCE — Essential

NotificationsModule + Notification model — Backend module + schema

A row is created whenever someone likes, comments, follows, or replies to a story — type, actor, target, and a read/unread flag.

USED IN — server/src/notifications/ and the Notification model in schema.prisma.

WHY — One unified table can represent every notification type without a table per type.

IMPORTANCE — Essential

NotificationsGateway — Backend, WebSocket

A NestJS WebSocket Gateway that pushes a new notification to a specific user's browser the instant it's created.

USED IN — server/src/notifications/notifications.gateway.ts, triggered from Likes/Comments/Follow services.

WHY — This is what makes the badge count update live instead of on refresh.

IMPORTANCE — Important — A v1 could just refetch every 30 seconds instead — real-time is an upgrade, not a hard requirement.

Read/unread state — Pattern

Notifications are marked read when the panel opens (or one is clicked), resetting the badge count.

USED IN — NotificationsService.markAsRead().

WHY — Without this the badge count would never go back down.

IMPORTANCE — Essential

---

shadcn/ui Popover / Sheet — Component library

Popover for a small desktop dropdown, Sheet for a slide-in panel on mobile — both handle positioning and focus-trapping.

USED IN — The notification panel component.

WHY — Notification panels have fiddly accessibility requirements that are easy to get wrong by hand.

IMPORTANCE — Good to know

---

7. Profile

Every user's own page — their posts, their bio, and the follow relationship if you're viewing someone else's.

Profile page — UI screen

Shows a user's avatar, bio, post count, follower/following counts, and a grid of their posts.

USED IN — app/(main)/profile/[username]/page.tsx — a dynamic route for any user's profile.

WHY — One of your four required sections, and the identity hub of the app.

IMPORTANCE — Essential

Edit profile modal — UI component

A form, visible only on your own profile, to update bio, avatar, and username.

USED IN — components/edit-profile-modal.tsx, opened only when viewing your own profile.

WHY — Users need a way to change what's on their profile after signup.

IMPORTANCE — Essential

Posts grid — Frontend component

The same grid-of-thumbnails pattern as Explore, filtered to this user's posts.

USED IN — Profile page, reusing the Explore grid layout logic.

WHY — Less code to maintain, and visually consistent with the rest of the app.

IMPORTANCE — Good to know

Follow / unfollow button + FollowModule — Backend module + UI

A toggle that creates or deletes a row in a Follow table (follower_id, following_id).

USED IN — server/src/follow/ on the backend; a button on the profile page reflecting current state.

WHY — This relationship is what the entire home feed query depends on.

IMPORTANCE — Essential — Not just a profile detail — foundational to the feed.

Followers / following list modal — UI component

A modal listing everyone who follows this user, or everyone this user follows.

USED IN — Profile page, opened by tapping the follower/following counts.

WHY — Lets people browse a user's network, same as the real app.

IMPORTANCE — Good to know

---

8. Chat (direct messages)

Real-time one-on-one messaging — where the app moves from a CRUD social app into genuinely real-time territory.

Chat list + chat window — UI screens

Chat list shows every conversation, most recent first; chat window is the open thread with message bubbles and an input.

USED IN — app/(main)/chat/page.tsx (list) and app/(main)/chat/[conversationId]/page.tsx (window).

WHY — The core DM experience you specifically asked for.

IMPORTANCE — Essential

ChatGateway — Backend, WebSocket

A NestJS WebSocket Gateway handling events like sendMessage, joinConversation, and typing.

USED IN — server/src/chat/chat.gateway.ts.

WHY — REST works for loading history, but a live conversation needs a persistent connection to push new messages instantly.

IMPORTANCE — Essential

---

socket.io-client — Library, frontend

The browser-side counterpart to NestJS's Socket.io gateways — opens and maintains the WebSocket connection.

USED IN — A small hook (e.g. useSocket()) that connects on login and stays open across the app.

WHY — Needed to actually receive the events ChatGateway, NotificationsGateway, and CallGateway emit.

IMPORTANCE — Essential

Conversation + Message models — Database schema

Conversation groups two users together; Message belongs to a conversation, with sender, content, and timestamps.

USED IN — schema.prisma.

WHY — Message history has to persist in Postgres — the WebSocket only handles delivery, not storage.

IMPORTANCE — Essential

Typing indicator — Pattern

A short-lived 'X is typing...' event, emitted while typing and cleared after a pause.

USED IN — ChatGateway's typing event, kept purely in-memory, never written to the database.

WHY — A small realism touch that makes chat feel alive.

IMPORTANCE — Good to know — Doesn't affect core functionality if skipped for v1.

Online/offline presence (Redis) — Pattern

Tracking which users have an active socket connection, usually as a set in Redis.

USED IN — ChatGateway's connect/disconnect handlers update Redis; the chat list reads it for a green dot.

WHY — Presence needs to be fast and ephemeral — a much better fit for Redis than Postgres.

IMPORTANCE — Good to know

Read receipts — Pattern

Marking a message as read when the recipient opens the conversation, reflected back to the sender.

USED IN — Message model's readAt field, updated when a conversation is opened.

WHY — Matches the 'seen' behavior people expect from modern messaging apps.

IMPORTANCE — Optional — A nice-to-have — chat works completely fine without it.

9. Video calling

The hardest part of the build. The server never actually sees the video or audio — it only helps two browsers find each other, then gets out of the way.

Call button + incoming-call modal — UI components

A button in the chat window to start a call, and a full-screen modal with accept/decline for the person being called.

USED IN — Chat window (call button); a global component (incoming-call modal) reachable from anywhere in the app.

WHY — Entry point for the whole calling feature.

IMPORTANCE — Essential

WebRTC (RTCPeerConnection) — Built-in browser API

A native browser API for direct peer-to-peer audio/video streaming between two browsers — no external library strictly required.

USED IN — A useWebRTC() hook or similar, used inside the call screen component.

WHY — The actual mechanism moving video/audio between the two callers once the connection is set up.

IMPORTANCE — Essential — The genuinely hard part of this project — budget real time to understand ICE candidates and the offer/answer exchange.

STUN / TURN servers — Infrastructure

STUN helps two devices behind routers discover their public address; TURN relays media when a direct connection isn't possible at all.

USED IN — Passed as config to RTCPeerConnection — a public STUN server (e.g. Google's) is free; TURN usually needs a small hosted service or a self-run coturn instance.

WHY — Without STUN/TURN, calls only work when both people are on the same network — basically never in the real world.

IMPORTANCE — Important — STUN alone gets most calls working; TURN is the fallback for harder network cases — fine to add second.

CallGateway (signaling) — Backend, WebSocket

Reuses the same WebSocket infrastructure as chat to pass offers, answers, and ICE candidates between the two callers before a direct connection exists.

USED IN — server/src/call/call.gateway.ts — can even live inside the chat gateway if you'd rather not split it out.

WHY — Two browsers can't find each other without some server introducing them first.

IMPORTANCE — Essential

Call screen UI — UI component

Full-screen video layout with mute, camera on/off, and end-call controls.

USED IN — components/call-screen.tsx.

WHY — Where the actual call happens once the connection is live.

IMPORTANCE — Essential

---

10. Cross-cutting infrastructure

The pieces that don't belong to any one page, but hold the whole thing together.

PostgreSQL — Database

The primary relational database — every user, post, like, comment, message, and story lives here.

USED IN — One database, reached by every NestJS module through Prisma.

WHY — Relational data (users following users, posts having many comments) fits a relational database naturally.

IMPORTANCE — Essential

Prisma ORM — Library

Defines your schema in one file (schema.prisma), generates a fully-typed client, and manages migrations.

USED IN — server/prisma/schema.prisma, and a PrismaService injected into every module that touches the database.

WHY — Writing raw SQL for every query gets unwieldy fast — Prisma gives type-safe queries matching your TypeScript setup.

IMPORTANCE — Essential

Redis — In-memory database

An in-memory store, used here for three things: feed caching, chat presence, and (optionally) scaling WebSockets across servers.

USED IN — A single Redis instance, connected via ioredis wherever it's needed.

WHY — Postgres is durable but relatively slow for these use cases; Redis is built for fast, temporary data.

IMPORTANCE — Important — Genuinely needed for presence/chat; the feed-caching use is closer to optional at small scale.

AWS S3 — Cloud storage

Object storage for every photo and video in the app — posts, stories, and avatars.

USED IN — One bucket, written to via presigned URLs from the UploadModule.

WHY — Databases are the wrong place for large binary files — S3 is built exactly for this.

IMPORTANCE — Essential

Docker + docker-compose — Dev tooling

Runs Postgres and Redis in containers locally, so your dev environment matches production.

USED IN — docker-compose.yml at the project root.

WHY — One command (docker compose up) gets a fresh setup fully running, no local installs needed.

IMPORTANCE — Important

@nestjs/config — Library

Loads environment variables (.env) into a typed, injectable config service instead of reading process.env directly.

USED IN — Imported once in AppModule, injected wherever a secret is needed (DB URL, JWT secret, S3 keys).

WHY — Keeps secrets out of your codebase and makes config changes a one-file affair.

IMPORTANCE — Important

CORS setup — Config

Explicitly allowing your Next.js frontend's origin to call your NestJS backend, since they run on different domains/ports.

USED IN — app.enableCors() in server/src/main.ts.

WHY — Without it the browser blocks every frontend-to-backend request by default.

IMPORTANCE — Essential — Easy to forget, and the resulting error is famously confusing the first time you hit it.

Global ValidationPipe + exception filters — Backend pattern

One line (app.useGlobalPipes(new ValidationPipe())) that applies your class-validator DTOs to every incoming request automatically.

USED IN — server/src/main.ts.

WHY — Without it you'd manually validate every request body by hand in every controller.

IMPORTANCE — Important

Zustand — Library

A tiny global state library for client-only state that doesn't come from the server — e.g. 'is a call currently active.'

USED IN — A few small stores, e.g. useCallStore, imported wherever that state is needed.

WHY — TanStack Query already covers anything from the server; Zustand fills the gap for purely client-side state.

IMPORTANCE — Good to know — Context API works fine too at this scale — Zustand is just less boilerplate.

Deployment targets — Infrastructure, no code

Vercel is the natural home for Next.js; the NestJS backend needs a host that supports persistent WebSocket connections (Render or Railway are common free-tier-friendly choices); Postgres and Redis are best as managed services (e.g. Neon/Supabase, Upstash) rather than self-hosted.

USED IN — N/A — an infrastructure decision, not code.

WHY — This is what makes 'fully functional and uploadable' actually true — a real URL other people can visit.

IMPORTANCE — Essential — Worth deciding early — WebSocket support isn't universal across free hosting tiers.

---

11. UI design system

The visual language used consistently across every page above — decided once, reused everywhere.

Tailwind CSS — Library

A utility-first CSS framework — you compose small utility classes directly in markup instead of writing separate CSS files.

USED IN — Every component in the Next.js app.

WHY — Fast to build with once you're used to it, and keeps styling co-located with the component.

IMPORTANCE — Essential

---

shadcn/ui — Component library

Not a typical npm package — a collection of accessible, unstyled-by-default components you copy directly into your codebase and customize freely.

USED IN — components/ui/, used across every page in this document.

WHY — Production-quality, accessible components without the black-box feeling of a heavier UI library.

IMPORTANCE — Essential

Radix UI — Underlying library

The low-level, unstyled primitives shadcn/ui is built on — handles the genuinely hard accessibility logic (focus trapping, keyboard nav, ARIA).

USED IN — Underneath shadcn/ui — rarely touched directly, but worth knowing it's there.

WHY — Explains why shadcn components behave correctly for keyboard and screen-reader users out of the box.

IMPORTANCE — Good to know

---

lucide-react — Library

The icon set shadcn/ui is built around — clean, consistent outline icons as React components.

USED IN — Anywhere an icon appears: heart, comment bubble, bell, search, camera, and more.

WHY — One consistent icon style across the whole app instead of mixing icon sets.

IMPORTANCE — Good to know

Typography & color tokens — Design decision

A small, deliberately limited set of font sizes, weights, and brand colors defined once, rather than picked freely per component.

USED IN — tailwind.config.ts.

WHY — Constraint is what makes a UI feel designed rather than assembled — also where your own branding shows up.

IMPORTANCE — Important — Worth deciding before building many screens — retrofitting later means touching every page.

Responsive design — Design decision

Making sure every page works from a narrow phone screen up to a wide desktop, using Tailwind's responsive prefixes (sm:, md:, lg:).

USED IN — Every component — most visibly the explore/profile grids and the chat layout.

WHY — A portfolio project people try on their phone shouldn't look broken there.

IMPORTANCE — Important

Appendix — full package reference

A quick-lookup cheat sheet of every package named in this document.

Frontend (Next.js)

Package	What it's for
next	The React framework itself — routing, rendering, image optimization.
react / react-dom	The underlying UI library Next.js is built on.
typescript	Static typing across the whole frontend.
tailwindcss	Utility-first styling.
shadcn/ui (CLI-managed)	Accessible component building blocks, copied into your codebase.
@tanstack/react-query	Server-state fetching and caching.
zustand	Small global client-state store.
react-hook-form	Form state management.
zod	Schema validation, paired with react-hook-form.
@hookform/resolvers	Connects zod schemas to react-hook-form.
socket.io-client	WebSocket connection to the NestJS gateways.
lucide-react	Icon set used throughout shadcn/ui.
date-fns	Relative timestamps, e.g. '2h ago'.

Backend (NestJS)

Package	What it's for
@nestjs/core, @nestjs/common	The NestJS framework itself.
@nestjs/config	Typed environment variable loading.
@nestjs/jwt, @nestjs/passport, passport	JWT issuing/verification and the strategy system.
passport-local, passport-jwt, passport-google-oauth20	The three login strategies: password, token check, Google.
bcrypt	Password hashing.
class-validator, class-transformer	DTO validation for every incoming request.

Package	What it's for
prisma, @prisma/client	Schema, migrations, and a type-safe database client.
ioredis	Redis client for caching and presence.
@nestjs/websockets, @nestjs/platform-socket.io	Powers ChatGateway, NotificationsGateway, and CallGateway.
cookie-parser	Reads the httpOnly auth cookie off incoming requests.
@aws-sdk/client-s3, @aws-sdk/s3-request-presigner	Generates presigned upload URLs for S3.
@nestjs/schedule (optional)	Cron-style cleanup, e.g. deleting expired stories.